

Variable-Structure Automaton Inference from Interaction Sequences

Engin Durmaz

May 15, 2026

Abstract

This report documents work toward inferring a Variable-Structure Automaton (VSA) from a unique dataset of state–action–feedback tuples generated in a P-model environment. We summarize the mathematical background for empirical counts and conditional estimates, the methodology for Tasks 1–4 (estimation, optimal-action identification, ε -optimality, and simulation) in mathematical terms, and outline planned work for Tasks 5–6. Implementation scripts live under `../proj_python/`; excerpts appear in Appendix A.

1 Introduction

The term project asks us to reconstruct, from observed sequences only, the state–action policy, transition dynamics, penalty structure, and norm of behavior of an unknown automaton with five states $\{\phi_1, \dots, \phi_5\}$, three actions $\{\alpha_1, \alpha_2, \alpha_3\}$, and binary environment feedback $\beta \in \{0, 1\}$ (P-model). Each dataset row is a tuple (`sequence_id`, `step`, $\Phi_{\text{src}}(n)$, $\Phi_{\text{dst}}(n)$, $\alpha(n)$, $\beta(n)$). This document aligns notation with the course assignment and with the implementation under `../proj_python/`.

2 Background

2.1 Empirical count $N(\phi_i, \alpha_k)$

The number of times action α_k was taken while the source state was ϕ_i is

$$N(\phi_i, \alpha_k) = \#\{n : \Phi_{\text{src}}(n) = \phi_i \wedge \alpha(n) = \alpha_k\}. \quad (1)$$

In the CSV, $\Phi_{\text{src}}(n)$ is the column `phi_src` and $\alpha(n)$ is `alpha`.

2.2 Policy estimate $P(\alpha_k \mid \phi_i)$

Empirical policy probabilities normalize the counts in each state:

$$P(\alpha_k \mid \phi_i) = \frac{N(\phi_i, \alpha_k)}{\sum_{k'} N(\phi_i, \alpha_{k'})} = \frac{\text{times } \alpha_k \text{ taken in } \phi_i}{\text{times } \phi_i \text{ was visited}}. \quad (2)$$

The denominator is the total number of actions chosen from state ϕ_i .

2.3 Row totals `row_totals_phi`

In code, for each state ϕ_i ,

$$\text{row_totals_phi}[\phi_i] = \sum_k N(\phi_i, \alpha_k), \quad (3)$$

i.e. the sum of the $N(\phi_i, \cdot)$ row in the count matrix `N_phi_alpha`.

2.4 Transition and penalty counts (brief)

Similarly, $N(\phi_i, \alpha_k, \phi_j)$ counts transitions $\phi_i \rightarrow \phi_j$ under action α_k , and $N(\alpha_k, \beta)$ counts (action, feedback) pairs. These yield transition probabilities $P(\phi_j \mid \phi_i, \alpha_k)$ and penalty probabilities $c_k = P(\beta = 1 \mid \alpha_k)$ by row-wise normalization of the appropriate count tables.

3 Methodology

All runnable code lives in `../proj_python/`. `load_all()` in `main.py` orchestrates tasks (calling the `run_*` entry points above); each task module can be run standalone (writing outputs under `outputs/`) or imported (returning structured results while still printing to the console). Excerpts of the Python modules for Tasks 1–4 are collected in Appendix A. Each task below ties the mathematics to concrete entry points (`run_*` functions) and to the corresponding appendix listing.

3.1 Task 1 — Estimation

We load the headerless UTF-8 CSV, build `N_phi_alpha` by grouping on `phi_src` and `alpha`, build per-action transition count matrices `N_phi_alpha_phi`, build `N_alpha_beta`, then derive `policy`, `transition`, and `penalty` (c_k). Let $r_i = \sum_k N(\phi_i, \alpha_k)$. For visited states ($r_i > 0$), $P(\alpha_k \mid \phi_i) = N(\phi_i, \alpha_k)/r_i$; for each action α_k , transition probabilities normalize each row of $N(\phi_i, \alpha_k, \phi_j)$ over ϕ_j . Penalties use $c_k = N(\alpha_k, 1) / \sum_{\beta \in \{0,1\}} N(\alpha_k, \beta)$, equivalently the empirical mean of β conditional on α_k . These operations realize the count-based policy in Section 2 and extend the same row-normalization idea to transition tables and to c_k from $N(\alpha_k, \beta)$. The companion code is in `estimation.py` (entry point `run_estimation`); shared types and column constants live in `util.py`. Appendix A.1 reproduces lines 117–164 (counts through `policy`, `transition`, `penalty`, and basic consistency checks).

3.2 Task 2 — Optimal action identification

Under the P-model, the optimal action minimizes the penalty probability:

$$\alpha^* \in \arg \min_k c_k, \quad c_k = P(\beta = 1 \mid \alpha_k). \quad (4)$$

The implementation reads c_k from the in-memory `EstimationResult` when called from `main.py`, or from `penalty.csv` after a standalone Task 1 run. Ties at the minimum are resolved deterministically (first label in sorted order). Computationally, $\alpha^* = \arg \min_k c_k$ among the empirical c_k from Task 1. The companion code is in `optimal_action_identification.py` (entry point `run_optimal_action`); Appendix A.2 shows lines 110–131 (tie-breaking at the minimum c_k and construction of the result object).

3.3 Task 3 — ε -optimality

Given α^* from Task 2 and the policy $P(\alpha \mid \phi)$ from Task 1, the assignment defines

$$\varepsilon = 1 - \min_i P(\alpha^* \mid \phi_i), \quad (5)$$

where the minimum is taken over **visited** states ϕ_i , i.e. rows of `N_phi_alpha` with positive row sum (states that appear in the data). States never visited are excluded so the minimum reflects empirical support. Let $\mathcal{V} = \{\phi_i : \sum_k N(\phi_i, \alpha_k) > 0\}$; then equivalently $\varepsilon = 1 - \min_{\phi_i \in \mathcal{V}} P(\alpha^* \mid \phi_i)$. The same quantity can be evaluated from stored matrices or from Task 1–2 outputs on disk. The companion code is in `e_optimality_analysis.py`; helper `_compute_core` evaluates the minimum over visited states, and `run_epsilon_optimality` dispatches in-memory `EstimationResult` / `OptimalActionResult` *vs.* CSV inputs from Tasks 1–2. Appendix A.3 lists lines 110–130.

3.4 Task 4 — Simulation

Synthetic trajectories use the learned **policy**, **transition** matrices, and **penalty** vector $c_k = P(\beta=1 \mid \alpha_k)$. For each real sequence ID we keep the **same length** and **same starting** Φ_{src} as in the dataset; each step samples $\alpha \sim P(\cdot \mid \phi)$, then $\phi' \sim P(\cdot \mid \phi, \alpha)$, then $\beta \sim \text{Bernoulli}(c_\alpha)$. Marginals of actions, ϕ_{src} , ϕ_{dst} , and penalty rates are compared to the real CSV; the code reports $|P_{\text{data}} - P_{\text{sim}}|$ per category (and L1-style sums). If a transition row is missing for the current ϕ , a uniform next-state distribution is used so sampling remains well-defined. Marginal distributions are empirical frequencies over rows (actions, Φ_{src} , Φ_{dst}); penalty comparison uses per-action sample means of β and the global mean. The companion code is in `simulation.py`; `_simulate_block` implements one synthetic dataset aligned with real sequence lengths and initial states, and `run_simulation` loads the real CSV, compares marginals, and aggregates absolute differences (from line 246 onward in the source file). Appendix A.4 shows lines 134–184.

3.5 Tasks 5–6 (planned)

- **Task 5 (perturbation)**: randomly remove 20% of transitions and re-estimate c_k , policy, and ε .
- **Task 6 (model critique)**: estimate c_k separately per source state and compare across states to assess the P-model assumption.

These will extend `load_all()` with new modules mirroring the dual-run convention used for Tasks 1–4.

4 Experiments

Data. The provided file `../Engin-vsa_dataset_full.csv` (one row per step; Greek state/action labels) is the default input for `estimation.py`.

Reproducibility.

1. Task 1 only (writes CSVs and `report.txt`): `cd ../proj_python && python estimation.py`
2. Task 2 after Task 1 files exist: `cd ../proj_python && python optimal_action_identification.py`
3. Task 3 after Tasks 1–2 outputs exist: `cd ../proj_python && python e_optimality_analysis.py`
4. Task 4 after Task 1 (and real CSV for marginals): `cd ../proj_python && python simulation.py`
5. End-to-end in memory (no intermediate files required): `cd ../proj_python && python main.py`

Planned extensions for the final report include the dataset fingerprint, full tables for Tasks 1–4, and (once implemented) perturbation analysis and per-state penalty comparison for the P-model critique.

5 Conclusion

We fixed notation for empirical counts and policy normalization, and linked Task 1 estimates to the count-based constructions in Section 2. Task 2 selects α^* by minimizing c_k ; Task 3 quantifies ε -optimality via $\min_i P(\alpha^* \mid \phi_i)$ over visited states; Task 4 validates the generative model against data marginals. Remaining work covers perturbation robustness and critique of the P-model assumption.

A Supplementary source code listing

In academic and technical writing it is common to relegate lengthy program text to an *appendix* (or to separate *supplementary material*) so the main body stays focused on definitions, assumptions, and results; journals often use the latter term for online-only addenda, while PDF reports typically use a lettered appendix. The excerpts below are line ranges from the course repository; they illustrate how the formulas in Section 3 are evaluated.

A.1 Task 1: estimation.py (probability construction)

```
# count by grouping how many action alpha valuated with beta
N_alpha_beta = (
    df.groupby([CLMN_ALPHA, CLMN_BETA]).size()
    .unstack(fill_value=0)
    .reindex(index=actions, columns=betas, fill_value=0)
)
N_alpha_beta.index.name = CLMN_ALPHA
N_alpha_beta.columns.name = CLMN_BETA

# Conditional probabilities -----
# N_phi_alpha : how many action alpha taken in state phi
# N_phi_alpha_phi : how many transitions from state phi_src to state phi_dst for action a
# N_alpha_beta : how many action alpha valuated with beta

row_totals_phi = N_phi_alpha.sum(axis=1)
policy = N_phi_alpha.div(row_totals_phi.replace(0, np.nan), axis=0).fillna(0.0)

transition: Dict[str, pd.DataFrame] = {}
for a in actions:
    N = N_phi_alpha_phi[a]
    T = N.div(N.sum(axis=1).replace(0, np.nan), axis=0).fillna(0.0)
    transition[a] = T

total_per_action = N_alpha_beta.sum(axis=1).replace(0, np.nan)
penalty = (N_alpha_beta[1] / total_per_action).fillna(0.0)
penalty.name = "c_k"

empirical_penalty = (
    df.groupby(CLMN_ALPHA)[CLMN_BETA].mean().reindex(actions, fill_value=0.0)
)
empirical_penalty.name = "empirical_penalty"

# Consistency checks -----
nonzero_phi = row_totals_phi > 0
policy_rows_ok = bool(
    np.allclose(policy.loc[nonzero_phi].sum(axis=1).values, 1.0, atol=1e-9)
)

transition_rows_ok = True
for a, T in transition.items():
    N = N_phi_alpha_phi[a]
    nonzero = N.sum(axis=1) > 0
    if not np.allclose(T.loc[nonzero].sum(axis=1).values, 1.0, atol=1e-9):
        transition_rows_ok = False
        break

penalty_matches_empirical = bool(
```

A.2 Task 2: optimal_action_identification.py

```
ties: List[str] = [str(a) for a in penalty.index[np.isclose(penalty.values, c_min)]]
alpha_star = ties[0] # deterministic pick: first action tied at the minimum

ranking = (
    penalty.sort_values(ascending=True)
        .to_frame()
        .assign(rank=lambda d: np.arange(1, len(d) + 1, dtype=int))
)
ranking["is_optimal"] = np.isclose(ranking["c_k"].values, c_min)
ranking.index.name = CLMN_ALPHA

result = OptimalActionResult(
    alpha_star=alpha_star,
    c_star=c_min,
    ties=ties,
    ranking=ranking,
    penalty=penalty,
    source=source,
)

# Human-readable report -----
lines: List[str] = []
```

A.3 Task 3: e_optimality_analysis.py (ϵ -optimality core)

```
def _compute_core(
    policy: pd.DataFrame,
    counts_phi_alpha: pd.DataFrame,
    alpha_star: str,
) -> Tuple[float, float, pd.Series, List[str]]:
    if alpha_star not in policy.columns:
        raise ValueError(
            f"alpha* column {alpha_star!r} not in policy columns {list(policy.columns)}."
            "Regenerate policy.csv / run Task 1 after Task 2."
        )
    row_totals = counts_phi_alpha.sum(axis=1)
    visited = row_totals > 0
    if not bool(visited.any()):
        raise ValueError("No visited states in count matrix; cannot compute epsilon.")

    p_col = policy.loc[visited, alpha_star].astype(float)
    min_p = float(np.min(p_col.values))
    epsilon = 1.0 - min_p
    tol = 1e-9
    argmin_states = [str(s) for s in p_col.index[np.isclose(p_col.values, min_p, atol=tol)]]
    return epsilon, min_p, p_col, argmin_states
```

A.4 Task 4: simulation.py (trajectory simulation)

```
def _simulate_block(
    real_df: pd.DataFrame,
    policy: pd.DataFrame,
    transition: Dict[str, pd.DataFrame],
    penalty: pd.Series,
    rng: np.random.Generator,
) -> pd.DataFrame:
    """Generate synthetic rows matching real (sequence_id, step) layout."""
    meta = (
```

```

real_df.sort_values([CLMN_SEQ_ID, CLMN_STEP])
.groupby(CLMN_SEQ_ID, sort=False)
.agg(length=(CLMN_STEP, "count"), phi0=(CLMN_PHI_SRC, "first"))
)
lengths = meta["length"].astype(int).to_numpy()
phi0s = meta["phi0"].astype(str).to_numpy()
seq_ids = meta.index.to_numpy()

states = list(policy.index.astype(str))
actions = list(policy.columns.astype(str))

rows: List[dict] = []
for sid, L, phi in zip(seq_ids, lengths, phi0s):
    phi = str(phi)
    if phi not in policy.index:
        phi = states[0]
    for step in range(1, int(L) + 1):
        prow = policy.loc[phi].to_numpy(dtype=float)
        a = _sample_from_row(rng, actions, prow)
        tmat = transition[a]
        labels = list(tmat.columns.astype(str))
        if phi in tmat.index:
            trow = tmat.loc[phi].to_numpy(dtype=float)
        else:
            trow = np.ones(len(labels), dtype=float) / len(labels)
        phi_next = _sample_from_row(rng, labels, trow)
        p_pen = float(penalty.reindex([a]).fillna(0.0).iloc[0])
        p_pen = min(max(p_pen, 0.0), 1.0)
        beta = int(rng.binomial(1, p_pen))
        rows.append(
            {
                CLMN_SEQ_ID: int(sid),
                CLMN_STEP: step,
                CLMN_PHI_SRC: phi,
                CLMN_PHI_DST: phi_next,
                CLMN_ALPHA: a,
                CLMN_BETA: beta,
            }
        )
    phi = phi_next

return pd.DataFrame(rows)

```